

SVD-ROM: A Python package for Singular Value Decomposition Reduced Order Modeling of large datasets.

David I. Salvador-Jasin¹, Robert Vava², Oliver Strickson¹, Louisa van Zeeland¹, Lydia France¹, Peter Yatsyshin¹, and Scott Hosking¹

¹ The Alan Turing Institute, London, United Kingdom ² Independent Researcher, London, United Kingdom  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

Despite their high dimensionality, datasets in fluid dynamics, weather, and climate often exhibit low-rank structure, where a small number of dominant patterns explain most of the variability. Singular Value Decomposition (SVD)-based methods, such as the Proper Orthogonal Decomposition (POD) ([Berkooz et al., 1993](#)), provide efficient and interpretable tools for dimensionality reduction in such systems. Dynamic Mode Decomposition (DMD) ([Schmid, 2022](#)) extends this framework to time-resolved data by extracting coherent spatio-temporal structures and their associated dynamics, which enables the construction of low-dimensional, interpretable emulators of complex dynamical systems. However, the wide-scale adoption of these methods in some domains, for example for weather and climate applications, has been limited by the challenges of applying them to large data volumes. We present SVD-ROM, an open-source Python package for SVD-based Reduced Order Modeling (ROM), designed to operate efficiently on large datasets using parallel and out-of-core computation on standard hardware.

Statement of need

Modern datasets in fields such as fluid dynamics, weather, and climate are characterized by high spatial and temporal resolution, typically leading to very large multi-dimensional data arrays that exceed the memory capacity of standard computing resources. Traditional SVD-based algorithms struggle to process such data due to memory constraints and computational bottlenecks. SVD-ROM addresses this challenge by providing a scalable framework that enables efficient computation of SVD-based reduced order models on datasets that would otherwise be intractable. This is achieved through a combination of scalable algorithms and efficient memory management and parallelization. In contrast to other packages, SVD-ROM is designed to operate on standard computing resources, such as laptop computers. It is purely written in Python and does not require specialized configuration or compilation. It is designed to be accessible to researchers and practitioners without requiring deep expertise in high-performance computing, making it a practical choice for ROM of large multi-dimensional arrays.

Related Work

Several open-source packages exist for ROM of dynamical systems, but most are either not designed for large-scale data or require specialized hardware. This section provides a brief overview of some notable packages in the field.

PyLDM ([Eiximeno et al., 2025](#)) is a Python library for ROM in fluid dynamics with algorithms tailored for supercomputers, where parallel operations are implemented using Message Passing

Interface (MPI). PySPOD (Mengaldo & Maulik, 2021) is a package for Spectral Proper Orthogonal Decomposition (SPOD) (Schmidt & Colonius, 2020), a frequency-domain extension of POD for statistically stationary time-resolved data, closely related to DMD (Towne et al., 2018). Similarly to PyLDM, PySPOD is designed for high-performance computing environments using MPI for parallelization. PyDMD (Ichinaga et al., 2024) is a Python package that implements DMD and several of its major variants. While it includes a number of cutting-edge methods developed to handle real-world data, it is not specifically designed to handle datasets larger than memory. SVD-ROM uses PyDMD internally to perform DMD at scale. pyMOR (Milk et al., 2016) is a library of ROM techniques for reducing the computational complexity of solving parametrized partial differential equation problems. While SVD-ROM is a data-centric/scalable ROM library, pyMOR is a model-centric ROM framework. Finally, EZyRB (Demo et al., 2018) is a general-purpose Python ROM framework focused on POD/reduced-basis methods and parametric model reduction, including interpolation of reduced-order solutions. While EZyRB provides a broader ROM toolbox than SVD-ROM, it is not specifically optimized for large-scale data processing.

Methods

Spatio-temporal data is arranged into an $(m \times n)$ snapshot matrix $\mathbf{X}$ whose rows index spatial locations and whose columns index time. Typical fluid dynamics applications have $m \gg n$ (the “tall-and-skinny” case), while weather and climate problems also have $m > n$ but with m exceeding n by a more moderate margin. Its SVD is $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$, where $\mathbf{U}$ and $\mathbf{V}$ hold the left and right singular vectors and $\mathbf{\Sigma}$ is the rectangular diagonal matrix whose entries are the singular values σ_j , ordered by decreasing magnitude. ROM proceeds by retaining only the leading k singular values, giving the rank k approximation $\mathbf{X}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^*$, which captures most of the variance of $\mathbf{X}$ when $k \ll n$.

POD is the direct application of the truncated SVD to the (optionally mean-removed) fluctuating field $\mathbf{X}'$, scaled by $1/\sqrt{n}$ so that modal energies do not depend on the number of snapshots:

$$\mathbf{X}'(\mathbf{x}, t) \approx \sum_{j=1}^k \phi_j(\mathbf{x}) a_j(t),$$

where the orthonormal spatial modes ϕ_j are the left singular vectors $\mathbf{U}_k$, the modal energies are $\lambda_j = \sigma_j^2$, and the time coefficients $a_j(t)$ follow from $\mathbf{\Sigma}_k \mathbf{V}_k^*$. Extended POD (Borée, 2003) correlates these modes with a second field $\mathbf{C}$ measured simultaneously in time, by projecting its fluctuating part onto the time coefficients as $\chi_j = (\lambda_j n)^{-1} \sum_i a_{ij} c'_i$, whose norm measures the energy in $\mathbf{C}'$ that is linearly correlated with mode j .

DMD (Schmid, 2022) complements this purely spatial reduction by also extracting temporal dynamics, seeking a decomposition $\mathbf{X} \approx \mathbf{\Phi}\mathbf{B}\mathbf{T}(\omega)$ into DMD modes $\mathbf{\Phi}$, amplitudes $\mathbf{B}$ and temporal dynamics $\mathbf{T}(\omega)$, whose j -th row is of the form $e^{\omega_j t}$ with ω_j the complex frequency (oscillation rate and growth or decay) governing the evolution of the j -th mode. Because these dynamics are continuous in time, they can be extrapolated beyond the training window to produce forecasts. SVD-ROM uses Optimized DMD (Askham & Kutz, 2018), which solves the exponential fitting problem directly by variable projection and is therefore robust to noise and to unevenly sampled snapshots. Crucially, the fit is performed in the SVD latent space,

$$\tilde{\mathbf{\Phi}}\tilde{\mathbf{B}}, \tilde{\omega} = \arg \min_{\tilde{\mathbf{\Phi}}\tilde{\mathbf{B}}, \tilde{\omega}} \|\mathbf{\Sigma}_k \mathbf{V}_k^* - \tilde{\mathbf{\Phi}}\tilde{\mathbf{B}}\mathbf{T}(\tilde{\omega})\|_F,$$

with the resulting $(k \times k)$ latent modes projected back to physical space via $\mathbf{\Phi} = \mathbf{U}_k \tilde{\mathbf{\Phi}}$. This encoder (truncated SVD) $\rightarrow$ processor (Optimized DMD) $\rightarrow$ decoder (orthogonal projection) framework allows SVD-ROM to fit DMD models on snapshot matrices that are far too large to be handled directly, and is the key contribution of this work.

84 In addition, it allows uncertainty quantification through bagging ([Sashidhar & Kutz, 2022](#)).

85 Software design

86 SVD-ROM is built on top of Xarray ([Hoyer & Hamman, 2017](#)) and Dask ([Rocklin, 2015](#)),
 87 and operates throughout on chunked, Dask-backed Xarray DataArray objects. This choice
 88 underpins both of the package's design goals. Firstly, labelled dimensions and coordinates
 89 mean that users work with named physical axes (for example, 'time', 'latitude' and 'longitude')
 90 rather than raw matrix indices, and are preserved across decomposition, reconstruction and
 91 forecasting. Secondly, the Dask backend provides the lazy, out-of-core and parallel execution
 92 that allows datasets larger than memory to be processed on a single machine, streaming from
 93 and to on-disk Zarr stores or NetCDF files.

94 All models share a common interface through an abstract `DecompositionModel` base class
 95 exposing the `fit` and `reconstruct` methods. The computational core is the `TruncatedSVD` class,
 96 which offers a choice between a deterministic, communication-efficient QR factorization for
 97 tall-and-skinny matrices ([Benson et al., 2013](#)), and a randomized algorithm ([Halko et al., 2011](#)),
 98 trading accuracy for speed and relaxed chunking requirements on very large arrays. POD extends
 99 `TruncatedSVD` with the appropriate energy normalization, mean removal and extended-POD
 100 functionality. `OptDMD` consumes the pre-computed SVD factors directly and solves the resulting
 101 low-dimensional nonlinear least-squares problem via the variable-projection Optimized DMD
 102 algorithm of the `PyDMD` package, with optional parallelized bootstrap aggregation for uncertainty
 103 quantification ([Sashidhar & Kutz, 2022](#)) and Hankel time-delay embedding ([Brunton et al., 2017](#))
 104 for systems with insufficient rank. The singular value decomposition is computed once
 105 and reused, meaning that a single decomposition can feed several downstream Optimized DMD
 106 models at no additional cost.

107 Throughout, users retain explicit control over what is materialized in memory: `compute_*`
 108 flags and companion methods make it possible to build a Dask computational graph and defer
 109 its evaluation, so that expensive quantities such as spatial modes or time coefficients are
 110 only computed when needed. The package is written in pure Python with no compilation or
 111 MPI configuration required, is fully type annotated, and ships optional extras for weather and
 112 climate workflows (`weather_utils`). An `SPOD` class is currently a placeholder, reflecting the
 113 intended extension of the same interface to further SVD-based methods in future releases.

114 Example

115 SVD-ROM fits a DMD model to a 36 GB slice of the ERA5 reanalysis dataset ([Hersbach et al., 2020](#))
 116 and forecasts 45 days beyond the training window in approximately 10 minutes on a
 117 10-core, 32 GB MacBook M1 Pro. The slice holds a single variable (temperature at the 500
 118 hPa pressure level) on a (time, latitude, longitude) grid: 8,766 snapshots at 4-hourly
 119 resolution between 1 Jan 2016 and 31 Dec 2019, on a 0.25° grid of $721 \times 1,440$ points, giving
 120 a $(1,038,240 \times 8,766)$ snapshot matrix.

```
import xarray as xr
```

```
from svdrom import OptDMD, TruncatedSVD
from svdrom.preprocessing import StandardScaler, variable_spatial_stack
```

```
# Open the store lazily, remove the time mean, and stack latitude and longitude
# into the "samples" dimension of a tall-and-skinny snapshot matrix.
```

```
X = xr.open_dataarray("era5_slice.zarr", chunks="auto")
scaler = StandardScaler()
X = variable_spatial_stack(scaler(X), dims=("latitude", "longitude")).T
```

```
# Encode with a randomized truncated SVD, then fit Optimized DMD on the triplet.
svd = TruncatedSVD(
```

```
n_components=40, algorithm="randomized", rechunk=True, compute_var_ratio=True
).fit(X, n_power_iter=2, n_oversamples=15)
dmd = OptDMD(n_modes=40, time_units="h").fit(svd.u, svd.s, svd.v)

# Forecast, keep the real part, restore the grid and the mean, and stream to disk.
forecast = dmd.forecast(forecast_span="45 D").real.unstack() + scaler.mean
forecast.to_zarr("forecast_45_days.zarr", mode="w")
```

Three SVD-ROM calls carry the whole workflow, and the data never leaves the labelled, chunked representation. Only the two fit calls and the mean removal are eager; everything else builds a Dask graph, so the forecast is returned on the original latitude/longitude grid with its time coordinate extended past the training window, and streams to Zarr chunk by chunk without ever being held in memory whole. Of the ten minutes, the randomized SVD accounts for roughly seven; mean removal, the DMD fit and the forecast take about one minute each. The 40 retained components explain over 80% of the variance of X , and the DMD fit operates on the resulting $(40 \times 8,766)$ latent triplet rather than on the full matrix, which is why it runs in-memory with a NumPy backend. Because that triplet is computed once and reused, further DMD models — at different ranks, or bagged for uncertainty quantification — have a small additional computational cost. The demos/ directory contains extended versions of this workflow, including retrieval of the ERA5 slice and validation of the forecast against climatology.

Research impact statement

SVD-ROM was developed as part of the Environmental Forecasting mission at the Alan Turing Institute, with the aim of making a computationally efficient and interpretable reduced order modeling and forecasting tool accessible to researchers and practitioners in fluid dynamics, weather, and climate. A major motivation was to enable the analysis of large datasets that are otherwise intractable on standard computing resources. SVD-ROM and related work has been presented at several conferences and workshops, including Climate Informatics 2026 in Lausanne (Salvador-Jasin et al., 2026), a PyData Meetup in London in February 2026 (Salvador-Jasin & Vava, 2026), and FOSDEM 2025 in Brussels (Salvador Jasin, 2025). The package is under active development and used by researchers at The Alan Turing Institute to explore use cases in climate and weather forecasting.

AI usage disclosure

Generative AI tools were used to assist with code development, to edit the documentation, and to review this manuscript. All AI-assisted output was reviewed, tested, and approved by the authors, who take full responsibility for the content of the software and this paper.

Acknowledgements

We acknowledge contributions from J. Nathan Kutz (Autodesk Research, UK) and Benet Eiximeno Franch (NVIDIA, Spain) for their advice on the design of a scalable DMD implementation, and from the PyDMD development team for their support in integrating Optimized DMD into SVD-ROM. Authors DSJ, OS, LvZ, LF, PY and SH were supported by The Alan Turing Institute.

References

Ashkam, T., & Kutz, J. N. (2018). Variable projection methods for an optimized dynamic mode decomposition. *SIAM Journal on Applied Dynamical Systems*, 17, 380–416. <https://doi.org/10.1137/M1124176>

- 159 Benson, A. R., Gleich, D. F., & Demmel, J. (2013). Direct QR factorizations for tall-and-skinny
160 matrices in MapReduce architectures. *2013 IEEE International Conference on Big Data*,
161 264–272. <https://doi.org/10.1109/BigData.2013.6691583>
- 162 Berkooz, G., Holmes, P., & Lumley, J. L. (1993). The proper orthogonal decomposition in
163 the analysis of turbulent flows [Journal Article]. *Annual Review of Fluid Mechanics*, 25,
164 539–575. <https://doi.org/10.1146/annurev.fl.25.010193.002543>
- 165 Borée, J. (2003). Extended proper orthogonal decomposition: A tool to analyse correlated
166 events in turbulent flows. *Experiments in Fluids*, 35, 188–192. <https://doi.org/10.1007/s00348-003-0656-3>
- 168 Brunton, S. L., Brunton, B. W., Proctor, J. L., Kaiser, E., & Kutz, J. N. (2017). Chaos
169 as an intermittently forced linear system. *Nature Communications*, 8(1), 19. <https://doi.org/10.1038/s41467-017-00030-8>
- 171 Demo, N., Tezzele, M., & Rozza, G. (2018). EZyRB: Easy Reduced Basis method. *The*
172 *Journal of Open Source Software*, 3(24), 661. <https://doi.org/10.21105/joss.00661>
- 173 Eiximeno, B., Miró, A., Begiashvili, B., Valero, E., Rodriguez, I., & Lehmkuhl, O. (2025).
174 PyLOM: A HPC open source reduced order model suite for fluid dynamics applications.
175 *Computer Physics Communications*, 308, 109459. <https://doi.org/10.1016/j.cpc.2024.109459>
- 177 Halko, N., Martinsson, P. G., & Tropp, J. A. (2011). Finding structure with randomness:
178 Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*,
179 53(2), 217–288. <https://doi.org/10.1137/090771806>
- 180 Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horányi, A., Muñoz-Sabater, J., Nicolas,
181 J., Peubey, C., Radu, R., Schepers, D., Simmons, A., Soci, C., Abdalla, S., Abellan,
182 X., Balsamo, G., Bechtold, P., Biavati, G., Bidlot, J., Bonavita, M., ... Thépaut, J.-N.
183 (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*,
184 146(730), 1999–2049. <https://doi.org/10.1002/qj.3803>
- 185 Hoyer, S., & Hamman, J. (2017). Xarray: N-d labeled arrays and datasets in python. *Journal*
186 *of Open Research Software*, 5(1), 10. <https://doi.org/10.5334/jors.148>
- 187 Ichinaga, S. M., Andreuzzi, F., Demo, N., Tezzele, M., Lapo, K., Rozza, G., Brunton, S. L., &
188 Kutz, J. N. (2024). PyDMD: A python package for robust dynamic mode decomposition.
189 *Journal of Machine Learning Research*, 25(417), 1–9. <http://jmlr.org/papers/v25/24-0739.html>
- 191 Mengaldo, G., & Maulik, R. (2021). PySPOD: A Python package for spectral proper orthogonal
192 decomposition (SPOD). *Journal of Open Source Software*, 6(60), 2862. <https://doi.org/10.21105/joss.02862>
- 194 Milk, R., Rave, S., & Schindler, F. (2016). pyMOR – generic algorithms and interfaces
195 for model order reduction. *SIAM Journal on Scientific Computing*, 38(5), S194–S216.
196 <https://doi.org/10.1137/15M1026614>
- 197 Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling.
198 *Proceedings of the 14th Python in Science Conference*, 126–132. <https://doi.org/10.25080/majora-7b98e3ed-013>
- 200 Salvador Jasin, D. (2025). *Explainable forecasting from big weather data: Rapid and sustainable*
201 *solutions*. Zenodo. <https://doi.org/10.5281/zenodo.14887704>
- 202 Salvador-Jasin, D., & Vava, R. (2026). *SVD-ROM: Reduced order modeling of huge arrays using*
203 *the singular value decomposition*. Zenodo. <https://doi.org/10.5281/zenodo.18468075>
- 204 Salvador-Jasin, D., Vava, R., Strickson, O., Zeeland, L. van, France, L., Eiximeno Franch,
205 B., Yatsyshin, P., Hosking, S., & Kutz, J. N. (2026). *SVD-ROM: Scalable reduced order*

- 206 *modeling of weather and climate data using the singular value decomposition*. Zenodo.
207 <https://doi.org/10.5281/zenodo.19710711>
- 208 Sashidhar, D., & Kutz, J. N. (2022). Bagging, optimized dynamic mode decomposition for
209 robust, stable forecasting with spatial and temporal uncertainty quantification. *Philosophical*
210 *Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*,
211 *380*(2229), 20210199. <https://doi.org/10.1098/rsta.2021.0199>
- 212 Schmid, P. J. (2022). Dynamic Mode Decomposition and Its Variants. *Annual Review of Fluid*
213 *Mechanics*, *54*, 225–254. <https://doi.org/10.1146/annurev-fluid-030121-015835>
- 214 Schmidt, O. T., & Colonius, T. (2020). Guide to spectral proper orthogonal decomposition.
215 *AIAA Journal*, *58*(3), 1023–1033. <https://doi.org/10.2514/1.J058809>
- 216 Towne, A., Schmidt, O. T., & Colonius, T. (2018). Spectral proper orthogonal decomposition
217 and its relationship to dynamic mode decomposition and resolvent analysis. *Journal of*
218 *Fluid Mechanics*, *847*, 821–867. <https://doi.org/10.1017/jfm.2018.283>

DRAFT